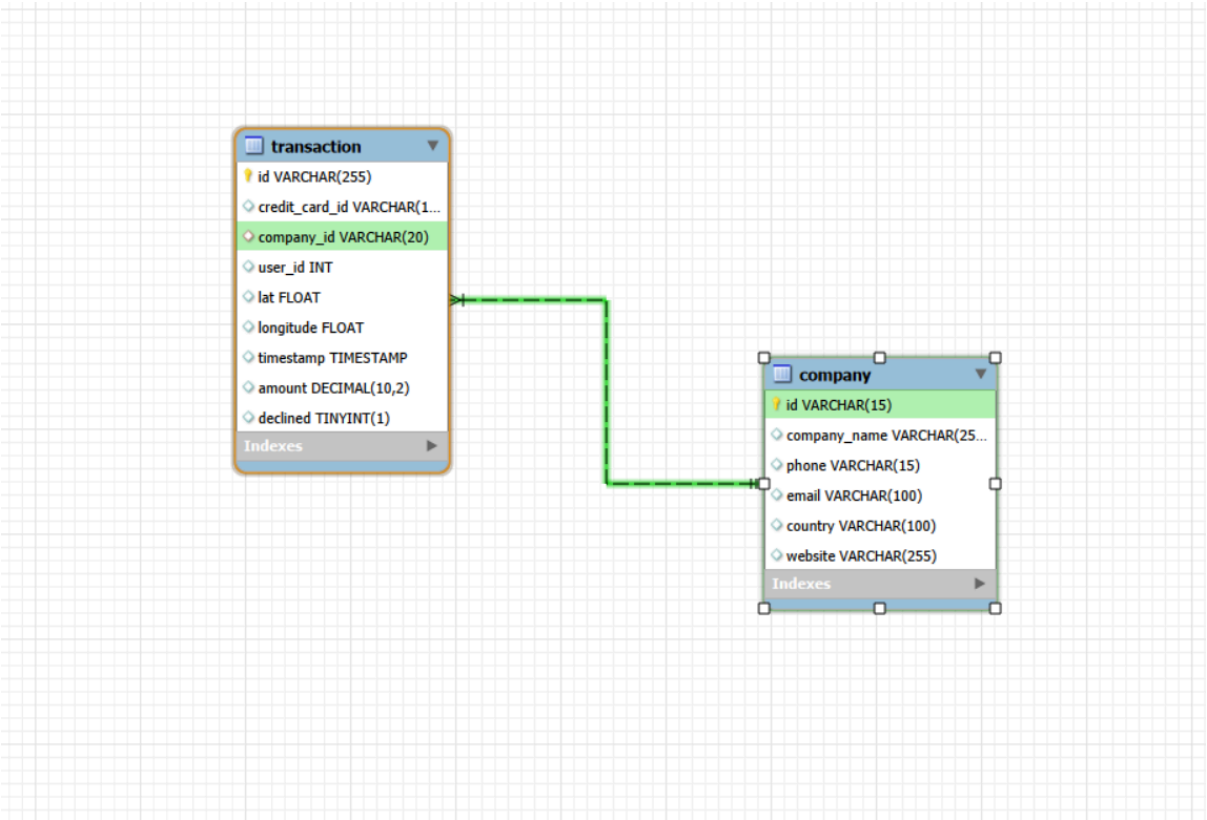
EJERCICIO 1

A partir de los documentos adjuntos (estructura_datos y datos_introducir), importa las dos tablas. Muestra las principales características del esquema creado y explica las diferentes tablas y variables que existen. Asegúrate de incluir un diagrama que ilustre la relación entre las distintas tablas y variables.



1. La tabla **company** tiene un id como clave primaria, esta es la “tabla de hechos” porque es independiente de la tabla **transaction**. **Company** contiene datos sobre las compañías como el nombre, teléfono, email, país, etc. Todos variables de tipo carácter.
2. La tabla **transaction** es de “dimensiones” y tiene un id como clave primaria también, más una clave foránea que es company_id que hace referencia a la tabla **company**. Además, contiene datos esenciales sobre las transacciones, como fecha del movimiento, cantidad, si fue declinada, etc.
3. Las tablas tienen una **relación 1:N (uno a muchos)** donde **company** (1) representa a una empresa que puede tener muchas transacciones (N).
4. La relación representa un **modelo estrella**.
5. Para la tabla **company** existen las siguientes variables:
    a. `id` **(VARCHAR(15))**
    Identificador único de la empresa. Es la **clave primaria (PRIMARY KEY)**, lo que significa que no se puede repetir ni ser nulo. Se define como texto de hasta 15 caracteres (por ejemplo, códigos alfanuméricos tipo "COMP12345").

b. `company_name` **(VARCHAR(255))**
Nombre de la empresa. Es un campo de texto que puede almacenar hasta 255 caracteres.
c. `phone` **(VARCHAR(15))**
d. Número de teléfono de la empresa. Se usa VARCHAR (texto) en lugar de un número porque puede incluir símbolos como +, espacios o guiones.
e. `email` **(VARCHAR(100))**
Correo electrónico de contacto de la empresa. Campo de texto con un máximo de 100 caracteres.
f. `country` **(VARCHAR(100))**
País donde se encuentra la empresa. Campo de texto de hasta 100 caracteres.
g. `website` **(VARCHAR(255))**
Sitio web de la empresa. Se almacena como texto para permitir URLs completas.

Todos los campos usan VARCHAR porque almacenan texto de longitud variable, lo cual es más eficiente que usar CHAR si el tamaño de los datos puede variar.

6. Para la tabla **transaction** existen aún más tipos de variables:
   a. `id` **(VARCHAR(255))**
   Nuevamente, un identificador único de la transacción. Es la **clave primaria**, así que no se repite ni puede ser nulo. Puede ser un ID largo o código alfanumérico.
   b. `credit_card_id` **(VARCHAR(15))**
   ID de la tarjeta de crédito usada en la transacción. Es una **clave foránea** que hace referencia a la tabla credit_card(id), conectando la transacción con una tarjeta específica.
   c. `company_id` **(VARCHAR(20))**
   ID de la empresa donde se realizó la transacción. También es **clave foránea**, enlazada con company(id).
   d. `user_id` **(INT)**
   ID del usuario que realizó la transacción. Es una **clave foránea** que apunta a user(id). Aquí sí es numérico (entero).
   e. `lat` **(FLOAT)**
   Latitud de la ubicación donde ocurrió la transacción. Es un número decimal aproximado.
   f. `longitude` **(FLOAT)**
   Longitud de la ubicación de la transacción. Igual que lat, es un número decimal.
   g. `timestamp` **(TIMESTAMP)**
   Fecha y hora exacta en la que ocurrió la transacción. Se usa mucho para análisis temporales (por ejemplo, picos de actividad).
   h. `amount` **(DECIMAL(10, 2))**

Monto de la transacción. Puede tener hasta 10 dígitos en total, con 2 decimales (por ejemplo, `12345678.99`). Ideal para dinero porque evita errores de precisión.

i. **`declined` (BOOLEAN)**
Indica si la transacción fue rechazada (TRUE) o aprobada (FALSE).

EJERCICIOS 2 / Utilizando JOINS:

a) Listado de los países que están generando ventas.

```
SELECT DISTINCT
    country
FROM company com
INNER JOIN transaction trans
    ON com.id = trans.company_id
WHERE declined = 0
ORDER BY country;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

country
Australia
Belgium
Canada
China
France
Germany
Ireland
Italy
Netherlands

Result 4 — Read Only

Output — Action Output

#	Time	Action	Message
1	21:44:19	SELECT DISTINCT country FROM company com INNER JOIN transaction trans ON com.id = trans.company...	15 row(s) returned

Aquí validé con la variable “declined” para asegurarme que solo se filtren los países que no tienen ventas declinadas, pues a mí me interesan los que procedieron correctamente (declined = 0, FALSE).

b) Desde cuántos países se generan las ventas.

```
SELECT
    COUNT(DISTINCT com.country) AS cuantos_paises
FROM company com
INNER JOIN transaction trans
    ON com.id = trans.company_id
WHERE declined = 0;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

cuantos_paises
15

Result 5 — Read Only

Output — Action Output

#	Time	Action	Message
1	21:44:19	SELECT DISTINCT country FROM company com INNER JOIN transaction trans ON com.id = trans.company...	15 row(s) returned
2	21:47:57	SELECT COUNT(DISTINCT com.country) AS cuantos_paises FROM company com INNER JOIN transactio...	1 row(s) returned

c) Identifica a la compañía con la mayor media de ventas.

```
SELECT
  ROUND(AVG(trans.amount), 2) AS media_ventas,
  com.company_name
FROM company com
INNER JOIN transaction trans
  ON com.id = trans.company_id
WHERE trans.declined = 0
GROUP BY com.id
ORDER BY media_ventas DESC
LIMIT 1;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: | Fetch rows:

media_ventas	company_name
284.91	Ac Fermentum Incorporated

Result 6 — Read Only

Output — Action Output

#	Time	Action	Message
1	21:44:19	SELECT DISTINCT country FROM company com INNER JOIN transaction trans ON com.id = trans.company...	15 row(s) returned
2	21:47:57	SELECT COUNT(DISTINCT com.country) AS cuantos_paises FROM company com INNER JOIN transactio...	1 row(s) returned
3	21:49:43	SELECT ROUND(AVG(trans.amount), 2) AS media_ventas, com.company_name FROM company com I...	1 row(s) returned

EJERCICIOS 3 / Utilizando solo subqueries:

a) Muestra todas las transacciones realizadas por empresas de Alemania.

```sql
-- Muestra todas las transacciones realizadas por empresas de Alemania.
SELECT trans.*, -- query externa
  (SELECT com.country -- subquery intermedia
    FROM company com
    WHERE com.id = trans.company_id) AS country
FROM transaction trans
WHERE EXISTS (SELECT com.id FROM company com WHERE com.id = trans.company_id AND com.country = 'Germany'); -- subquery interna
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: | Fetch rows:

id	credit_card_id	company_id	user_id	lat	longitude	timestamp	amount	declined	country
0E71CF3									
0E96910									
0EB7170									
0EB9F4C7-1299-440A-A463-6660AC79C403	CcS-7001	b-2222	2420	48.8518	2.34317	2019-07-31 09:42:34	14.10	0	Germany
0EC9B9AA-9687-4614-86F2-F32811CEDC4D	CcS-6511	b-2222	1930	40.1976	-4.02352	2016-01-17 08:37:19	326.68	0	Germany
0EEF3ED8-0D98-4F47-BD1B-B520D3AB89CF	CcS-6585	b-2222	2004	51.9266	19.0042	2022-10-08 04:44:14	590.95	0	Germany
0EF31A60-6221-4BEF-885B-43935857FCCD	CcS-8714	b-2222	4133	59.3389	18.065	2016-04-16 16:02:49	301.33	0	Germany
0EFC8965-F917-4C8B-8F10-276B6909F340	CcS-4969	b-2222	388	41.391	2.16525	2021-09-20 10:00:04	295.67	0	Germany
0F05617D-1A17-444C-AFCD-5B07FF12F69C	CcS-5024	b-2222	443	39.271	-8.46075	2021-04-28 13:26:51	259.49	0	Germany

Result 8 | Read Only

Output

Action Output

#	Time	Action	Message
1	21:56:33	SELECT trans.*, -- query externa (SELECT com.country -- subquery intermedia FROM company com W...	1000 row(s) returned
2	21:58:52	SELECT trans.*, -- query externa (SELECT com.country -- subquery intermedia FROM company com W...	1000 row(s) returned

b) Lista las empresas que han realizado transacciones por un amount superior a la media de todas las transacciones.

```sql
-- Lista las empresas que han realizado transacciones por un amount superior a la media de todas las transacciones.
SELECT company_name
FROM company c
WHERE EXISTS (
    SELECT 1
    FROM transaction t
    WHERE t.company_id = c.id
      AND t.amount > (SELECT AVG(amount) FROM transaction)
);
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

company_name
Ac Fermentum Incorporated
Magna A Neque Industries
Fusce Corp.
Convallis In Incorporated
Ante Iaculis Nec Foundation
Donec Ltd
Sed Nunc Ltd
Amet Nulla Donec Corporation
Nascetur Ridiculus Mus Inc.

company 10 | Read Only

Output

Action Output

#	Time	Action	Message
1	22:41:26	SELECT company_name FROM company c WHERE EXISTS (SELECT 1 FROM transaction t WHE...	100 row(s) returned

c) Eliminarán del sistema las empresas que carecen de transacciones registradas, entrega el listado de estas empresas.

```
99  SELECT com.id AS empresas_sin_transacc -- Primero identifico cuales serian esas empresas que no tienen registros en transacciones por eso
100 FROM company com
101 WHERE NOT EXISTS (
102   SELECT 1 -- used to check whether a record matching your where clause exists.
103   FROM transaction trans
104   WHERE trans.company_id = com.id
105 );
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

empresas_sin_transacc

company 12 × Read Only

Output

Action Output

#	Time	Action	Message
1	22:50:07	SELECT com.id AS empresas_sin_transacc -- Primero identifico cuales serian esas empresas que no tienen re...	0 row(s) returned

No hay empresas.
Se usaría la declaración “delete from” para eliminar definitivamente las empresas.

EJERCICIO 4

a) *Diseñar y crear una tabla llamada "credit_card" que almacene detalles cruciales sobre las tarjetas de crédito.*

   La nueva tabla debe ser capaz de identificar de forma única cada tarjeta y establecer una relación adecuada con las otras dos tablas ("transaction" y "company"). Después de crear la tabla será necesario que ingreses la información del documento denominado "datos_introducir_credit". Recuerda mostrar el diagrama y realizar una breve descripción del mismo.

```sql
-- EJERCICIO 4 (Nueva tabla)
-- Crear nueva tabla credit_card
CREATE TABLE IF NOT EXISTS credit_card (
id VARCHAR(15) PRIMARY KEY,
iban VARCHAR(34) NOT NULL,
pan CHAR(16) NOT NULL,
pin CHAR(4) NOT NULL,
cvv VARCHAR(4) NOT NULL,
expiring_date VARCHAR(10) NOT NULL
);
```

Output

Action Output

#	Time	Action	Message
1	22:50:07	SELECT com.id AS empresas_sin_transacc -- Primero identifico cuales serian esas empresas que no tienen re...	0 row(s) returned
2	23:19:25	CREATE TABLE IF NOT EXISTS credit_card (id VARCHAR(15) PRIMARY KEY, iban VARCHAR(34) NOT ...	0 row(s) affected

Luego modifico la tabla **transaction** para agregarle una clave foránea que haga referencia a **credit_card_id**.

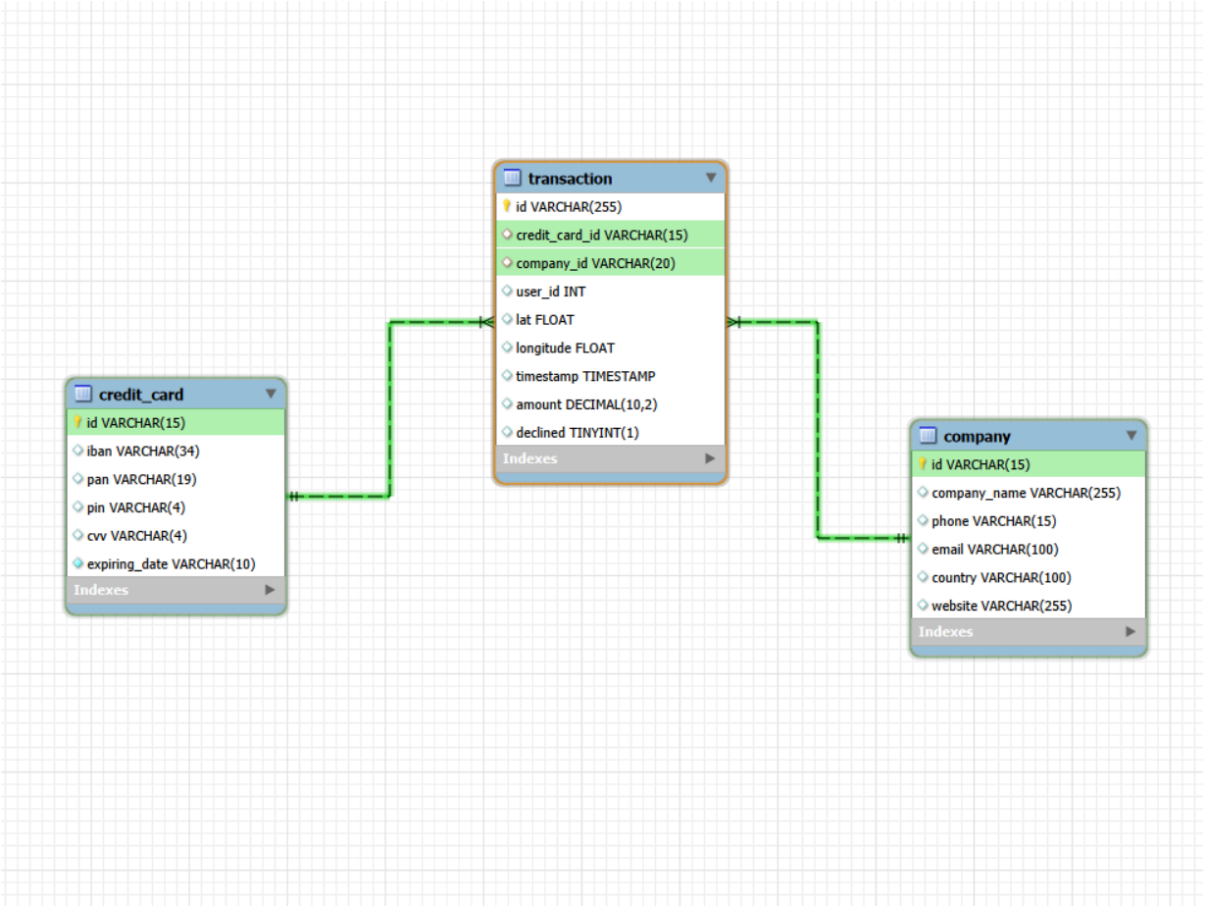
```sql
ALTER TABLE transaction -- Modifico transaction para crear la Foreign Key y para que sea de manera segura, uso constraint
ADD CONSTRAINT fk_trans_cc_01
FOREIGN KEY (credit_card_id) REFERENCES credit_card(id);

-- Modificando la tabla credit_card, me aseguro que pan acepta los espacios
ALTER TABLE credit_card
MODIFY pan VARCHAR(19);

UPDATE credit_card
SET expiring_date = STR_TO_DATE(expiring_date, '%m/%d/%y'); -- modificación de la fecha para que esté en el formato adecuado (date)
```

Output

Action Output

#	Time	Action	Message
5011	23:59:06	SELECT * FROM credit_card LIMIT 5	5 row(s) returned
5012	23:59:30	SELECT DISTINCT credit_card_id FROM transaction WHERE credit_card_id IS NOT NULL AND credit_...	0 row(s) returned
5013	00:00:08	ALTER TABLE transaction ADD CONSTRAINT fk_trans_cc_01 FOREIGN KEY (credit_card_id) REFERE...	100000 row(s) affected Records: 100000 Duplicates: 0 W
5014	00:01:00	-ALTER TABLE credit_card MODIFY pan VARCHAR(19)	Error Code: 1064. You have an error in your SQL syntax; ch
5015	00:01:08	ALTER TABLE credit_card MODIFY pan VARCHAR(19)	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0
5016	00:01:30	UPDATE credit_card SET expiring_date = STR_TO_DATE(expiring_date, '%m/%d/%y')	5000 row(s) affected Rows matched: 5000 Changed: 5000

Herramienta Recor
Captura de pantalla
Guardado automátic
capturas de pantalla
Marcac



Aquí aparece la nueva tabla **"credit_card"** que tiene como clave primaria "id" y **transaction** se relaciona con "credit_card_id" (llave foránea). Se mantiene un esquema tipo estrella porque la tabla **transaction** es la tabla de hechos mientras que **company** y **credit_card** son tablas de dimensión que describen y dan atributos a las transacciones.

EJERCICIO 5

El departamento de Recursos Humanos ha identificado un error en el número de cuenta asociado a su tarjeta de crédito con ID CcU-2938. La información que debe mostrarse para este registro es: TR323456312213576817699999. Recuerda mostrar que el cambio se realizó.

```
137 •  SELECT * -- primero reviso que sí está el id requerido y corroboro que el iban es distinto
138    FROM credit_card
139    WHERE id='CcU-2938';
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

id	iban	pan	pin	cvv	expiring_date
CcU-2938	TR3019503122135768176386 61	5424465566813633	3257	984	2022-10-30
NULL	NULL	NULL	NULL	NULL	NULL

credit_card 21

Output

Action Output

#	Time	Action	Message
1	00:07:09	SELECT * -- primero reviso que sí está el id requerido y corroboro que el iban es distinto FROM credit_card W...	1 row(s) returned

---

Después con el update:

```
137 •  SELECT * -- primero reviso que sí está el id requerido y corroboro que el iban es distinto
138    FROM credit_card
139    WHERE id='CcU-2938';
140
141 •  UPDATE credit_card SET iban ='TR3234563122135768176999 99' WHERE id ='CcU-2938';
142
143
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

id	iban	pan	pin	cvv	expiring_date
CcU-2938	TR3234563122135768176999 99	5424465566813633	3257	984	2022-10-30
NULL	NULL	NULL	NULL	NULL	NULL

credit_card 22

Output

Action Output

#	Time	Action	Message
1	00:07:09	SELECT * -- primero reviso que sí está el id requerido y corroboro que el iban es distinto FROM credit_card W...	1 row(s) returned
2	00:08:11	UPDATE credit_card SET iban ='TR3234563122135768176999 99' WHERE id ='CcU-2938'	1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0
3	00:08:14	SELECT * -- primero reviso que sí está el id requerido y corroboro que el iban es distinto FROM credit_card W...	1 row(s) returned

EJERCICIO 6

En la tabla "transaction" ingresa una nueva transacción con la siguiente información:

```
Id

108B1D1D-5B23-A76C-55EF-C568E49A99DD

credit_card_id

CcU-9999

company_id

b-9999

user_id

9999

lat

829.999

longitude

-117.999

amount

111.11

declined

0
```

```
-- En la tabla "transaction" ingresa una nueva transacción con la siguiente información: (imagen)
INSERT INTO transaction (id, credit_card_id, company_id, user_id, lat, longitude, amount, declined) -- registro nuevo
VALUES ('108B1D1D-5B23-A76C-55EF-C568E49A99DD', 'CcU-9999', 'b-9999', 9999, 829.999, -117.999, 111.11, 0);

SELECT *
FROM transaction
WHERE credit_card_id = 'CcU-9999';
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

id	credit_card_id	company_id	user_id	lat	longitude	timestamp	amount	declined
108B1D1D-5B23-A76C-55EF-C568E49A99DD	CcU-9999	b-9999	9999	829.999	-117.999	NULL	111.11	0
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

transaction 27

Output

Action Output

#	Time	Action	Message
1	00:18:04	INSERT INTO transaction (id, credit_card_id, company_id, user_id, lat, longitude, amount, declined) -- registro...	1 row(s) affected
2	00:18:52	SELECT * FROM transaction WHERE company_id = 'CcU-9999' LIMIT 0, 1000	0 row(s) returned
3	00:20:10	SELECT * FROM transaction ORDER BY timestamp DESC LIMIT 1	1 row(s) returned
4	00:20:26	INSERT INTO transaction (id, credit_card_id, company_id, user_id, lat, longitude, amount, declined) -- registro...	Error Code: 1062. Duplicate entry '108B1D1D-5B23-A76C-55EF-C568E49A99DD' for k
5	00:21:11	SELECT * FROM transaction WHERE credit_card_id = 'CcU-9999' LIMIT 0, 1000	1 row(s) returned

EJERCICIO 8

Desde recursos humanos te solicitan eliminar la columna "pan" de la tabla credit_card. Recuerda mostrar el cambio realizado.

```
-- Desde recursos humanos te solicitan eliminar la columna "pan" de la tabla credit_card. Recuerda mostrar el cambio realizado.
ALTER TABLE credit_card
DROP COLUMN pan;

SELECT *
FROM credit_card
LIMIT 5;
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content: | Fetch rows:

id	iban	pin	cvv	expiring_date
CcS-4857	XX4857591835292505850771	1819	467	2025-09-27
CcS-4858	XX8581768137002436094025	3964	817	2028-12-28
CcS-4859	XX7826930491423553609370	4983	277	2026-11-26
CcS-4860	XX5559590368835304645299	6876	661	2027-07-27
CcS-4861	XX2035182877195191627307	5710	398	2026-04-25
NULL	NULL	NULL	NULL	NULL

credit_card 28

Output

Action Output

#	Time	Action	Message
1	00:22:43	ALTER TABLE credit_card DROP COLUMN pan	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0
2	00:22:51	SELECT * FROM credit_card LIMIT 5	5 row(s) returned

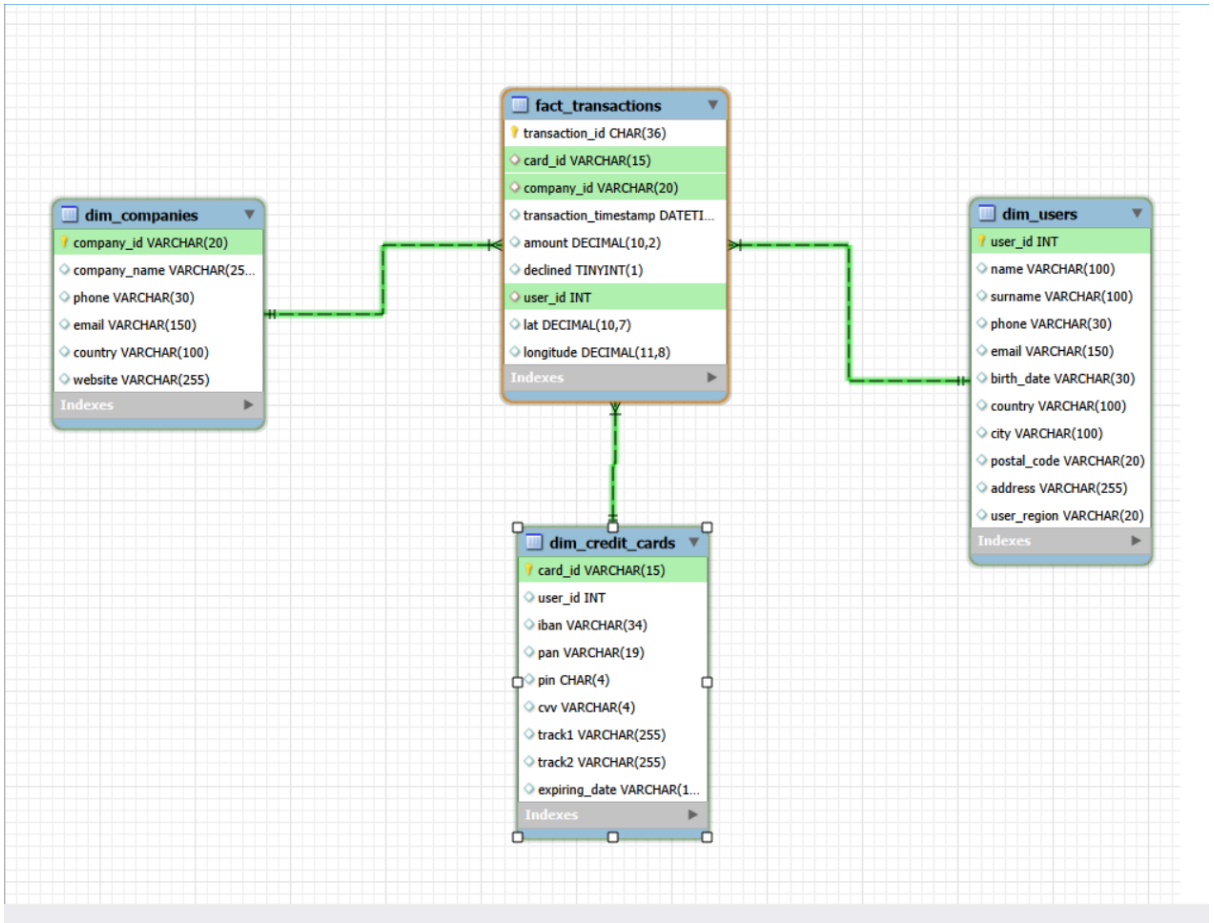
EJERCICIO 8

Descarga los archivos CSV que encontrarás en el apartado de recursos:

american_users.csv

european_users.csv
companies.csv
credit_cards.csv
transactions.csv

Estudia y diseña una base de datos con un esquema de estrella que contenga, al menos 4 tablas de las que puedas realizar las siguientes consultas:



Decidí nombrar la base de datos como **user_transactions**. Esta sigue un esquema de estrella donde la tabla central **fact_transactions** almacena los hechos transaccionales, como monto, fecha, rechazo y ubicación. Esta tabla se relaciona con las dimensiones **dim_users** (uní a los usuarios de Europa y Estados Unidos para evitar redundancia y mantener el modelo más limpio), **dim_companies** y **dim_credit_cards**, que aportan contexto descriptivo para el análisis. Las relaciones son de **uno a muchos** desde cada dimensión hacia la tabla de hechos.

Declaración de las claves foráneas al crear la tabla de hechos **transactions**:

```sql
CREATE TABLE fact_transactions (
    transaction_id CHAR(36) PRIMARY KEY,
    card_id VARCHAR(15),
    company_id VARCHAR(20),
    transaction_timestamp DATETIME,
    amount DECIMAL(10,2),
    declined TINYINT(1),
    user_id INT,
    lat DECIMAL(10,7),
    longitude DECIMAL(11,8),

    CONSTRAINT fk_fact_card
        FOREIGN KEY (card_id) REFERENCES dim_credit_cards(card_id),

    CONSTRAINT fk_fact_company
        FOREIGN KEY (company_id) REFERENCES dim_companies(company_id),

    CONSTRAINT fk_fact_user
        FOREIGN KEY (user_id) REFERENCES dim_users(user_id)
);
```

```sql
LOAD DATA LOCAL INFILE 'C:/Users/conta/Downloads/N1.Ex.8__ european_users.csv'
INTO TABLE dim_users
FIELDS TERMINATED BY ','
ENCLOSED BY '"'
LINES TERMINATED BY '\n'
IGNORE 1 ROWS -- ignora la primera fila (header)
(user_id, name, surname, phone, email, birth_date, country, city, postal_code, address)
SET user_region = 'Europe';

-- -------------------------
-- USUARIOS AMERICANOS
-- -------------------------
LOAD DATA LOCAL INFILE 'C:/Users/conta/Downloads/N1-Ex.8__ american_users.csv'
INTO TABLE dim_users
FIELDS TERMINATED BY ','
ENCLOSED BY '"'
LINES TERMINATED BY '\n'
IGNORE 1 ROWS
(user_id, name, surname, phone, email, birth_date, country, city, postal_code, address)
SET user_region = 'America';
```

Al cargar los archivos, agregué la columna “user_region” para todos los registros de Europa y luego para todos los de America, así no perdí esa información del dataset.

```sql
-- -------------------------
-- TRANSACCIONES
-- este CSV usa ;
-- product_ids se ignora con variable temporal
-- -------------------------
LOAD DATA LOCAL INFILE 'C:/Users/conta/Downloads/N1.Ex.8__ transactions.csv'
INTO TABLE fact_transactions
FIELDS TERMINATED BY ';'
ENCLOSED BY '"'
LINES TERMINATED BY '\n'
IGNORE 1 ROWS
(transaction_id, card_id, company_id, transaction_timestamp, amount, declined, @product_ids, user_id, lat, longitude);
```

Para transactions tuve que modificar el tipo de separación de cada campo, que en este archivo usaba “;” en lugar de “,”. También agregué el “@” en product_ids para leer el valor

de esa columna del archivo, pero no insertarlo directamente en la tabla. Se guarda temporalmente en @product_ids

EJERCICIO 9

Realiza una subconsulta que muestre a todos los usuarios con más de 80 transacciones utilizando al menos 2 tablas.

```
155 •  SELECT
156        diu.user_id,
157        diu.name,
158        diu.surname
159    FROM dim_users diu
160    WHERE diu.user_id IN (
161        SELECT user_id
162        FROM fact_transactions
163        GROUP BY user_id
164        HAVING COUNT(*) > 80
165    );
166
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

user_id	name	surname
185	Molly	Gilliam
289	Dxwgi	Hwcru
318	Bnyr	Astuw
454	Sfzzoh	Xgvfridxs
NULL	NULL	NULL

Result Grid | Form Editor | Apply

dim_users 3

Output

Action Output

#	Time	Action	Message
25	11:56:11	LOAD DATA LOCAL INFILE 'C:/Users/conta/Downloads/N1-Ex.8__american_users.csv' INTO TABLE...	1010 row(s) affected Records: 1010 Deleted: 0 Skipped: 0 Warnings: 0
26	11:56:11	LOAD DATA LOCAL INFILE 'C:/Users/conta/Downloads/N1.Ex.8__companies.csv' INTO TABLE dim...	100 row(s) affected Records: 100 Deleted: 0 Skipped: 0 Warnings: 0
27	11:56:11	LOAD DATA LOCAL INFILE 'C:/Users/conta/Downloads/N1.Ex.8__credit_cards.csv' INTO TABLE di...	5000 row(s) affected Records: 5000 Deleted: 0 Skipped: 0 Warnings: 0
28	11:56:11	LOAD DATA LOCAL INFILE 'C:/Users/conta/Downloads/N1.Ex.8__transactions.csv' INTO TABLE fac...	
29	11:56:25	SELECT diu.user_id, diu.name, diu.surname FROM dim_users diu WHERE diu.user_id IN (S...	4 row(s) returned
30	11:56:25	SELECT cc.iban, ROUND(AVG(amount), 2) AS media_amount, dc.company_name FROM fact_tran...	371 row(s) returned
31	12:05:12	SELECT diu.user_id, diu.name, diu.surname FROM dim_users diu WHERE diu.user_id IN (S...	4 row(s) returned

EJERCICIO 10

Muestra la media de amount por IBAN de las tarjetas de crédito en la compañía Donec Ltd., utiliza por lo menos 2 tablas.

```sql
SELECT
    cc.iban,
    ROUND(AVG(amount), 2) AS media_amount,
    dc.company_name
FROM fact_transactions fa
JOIN dim_credit_cards cc
    ON fa.card_id = cc.card_id
JOIN dim_companies dc
    ON fa.company_id = dc.company_id
WHERE dc.company_name = 'Donec Ltd'
GROUP BY cc.iban; -- b-2242
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

iban	media_amount	company_name
XX9114064011255863075868 05	356.25	Donec Ltd
SK9446370242474562577506	142.96	Donec Ltd
XX7767529178459529755556 40	257.37	Donec Ltd
XX4138273622897193049089 90	139.59	Donec Ltd
XX3477872460707696107803 08	240.41	Donec Ltd
XX6887684365430908948546 02	188.58	Donec Ltd
MK2836885153868834 9	439.39	Donec Ltd
PL7624928356685267634340 4576	541.56	Donec Ltd
XX8040083311349183418039 13	189.21	Donec Ltd

Result 1 | Read Only

Output

Action Output

#	Time	Action	Message
1	00:37:02	SELECT cc.iban, ROUND(AVG(amount), 2) AS media_amount, dc.company_name FROM fact_transactio...	371 row(s) returned

NIVEL 2

EJERCICIO 1

Identifica los cinco días que se generó la mayor cantidad de ingresos en la empresa por ventas. Muestra la fecha de cada transacción junto con el total de las ventas.

```sql
SELECT
    DATE(transaction_timestamp) AS fecha,
    SUM(amount) AS total_ventas
FROM fact_transactions
WHERE declined = 0
GROUP BY DATE(transaction_timestamp)
ORDER BY total_ventas DESC
LIMIT 5;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: | Fetch rows:

fecha	total_ventas
2022-12-13	14337.44
2019-11-18	13591.32
2023-02-20	13332.59
2017-12-20	13318.43
2019-03-18	12680.95

Result 3 | Read Only

Output

Action Output

#	Time	Action	Message
1	00:45:06	SELECT DATE(transaction_timestamp) AS fecha, SUM(amount) AS total_ventas FROM fact_transactio...	5 row(s) returned